

Frequency Modulated Continuous Wave Project Report

Determine the range, angular position and velocity



South University Of Science And Technology
Signal And System
SME 206

12111315 Li Zhenghao
12211640 Guo Yongming

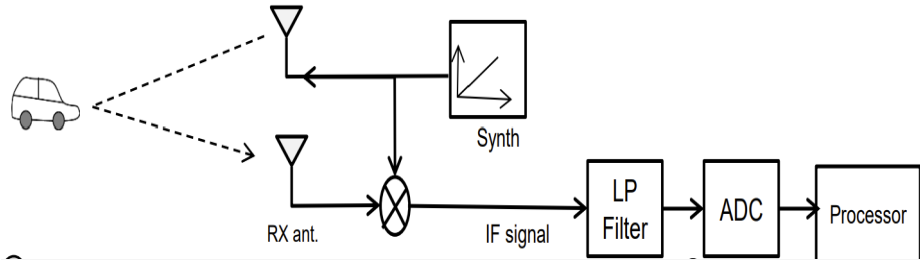
January 2024

Contents

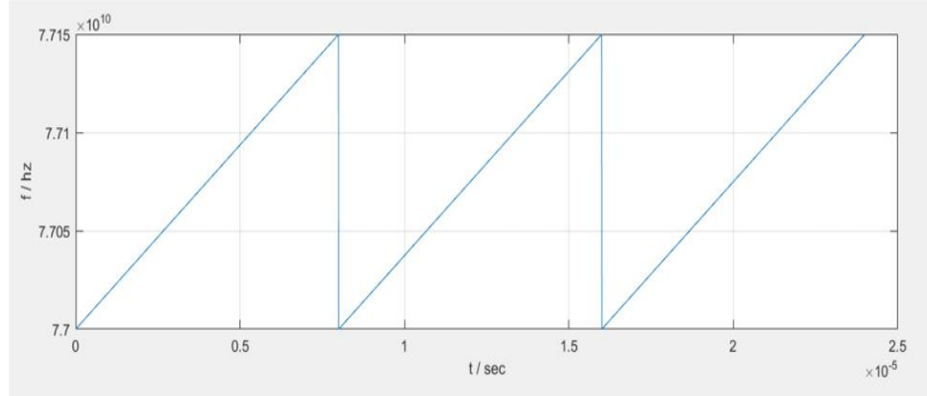
1	FMCW Radar Background Introduction	1
2	Analysis	2
2.1	Electric Circuit	2
2.2	Theory Analysis	5
3	Plot And Result	9
3.1	Question A	9
3.1.1	Range Result	9
3.2	Question B	10
3.2.1	IF Signal	10
3.2.2	FFT Result	10
3.2.3	FFT2 Result	11
3.2.4	Velocity Result	11
3.3	Question C	12
3.3.1	Range Resolution	12
3.3.2	Velocity Resolution	12
3.4	Question D	13
3.4.1	System	13
3.4.2	IF Signal	14
3.4.3	FFT Result	15
3.4.4	FFT2 Result	16
3.4.5	Velocity Result	16
3.5	Question E	17
4	Reflection	18
5	Extension	19
6	Code	20
6.1	Question A	20
6.2	Question B	21
6.3	Question D	23
6.4	Question E	26

1 FMCW Radar Background Introduction

FMCW radars transmit a frequency-modulated signal continuously in order to measure range as well as angle and velocity. This differs from traditional pulsed-radar systems, which transmit short pulses periodically. The modularized show of the system is the follow plot.



An FMCW radar transmits a signal called a “chirp”. A chirp is a sinusoid whose frequency increases linearly with time, as shown in the Frequency vs time (or ‘F-t’ plot) here.



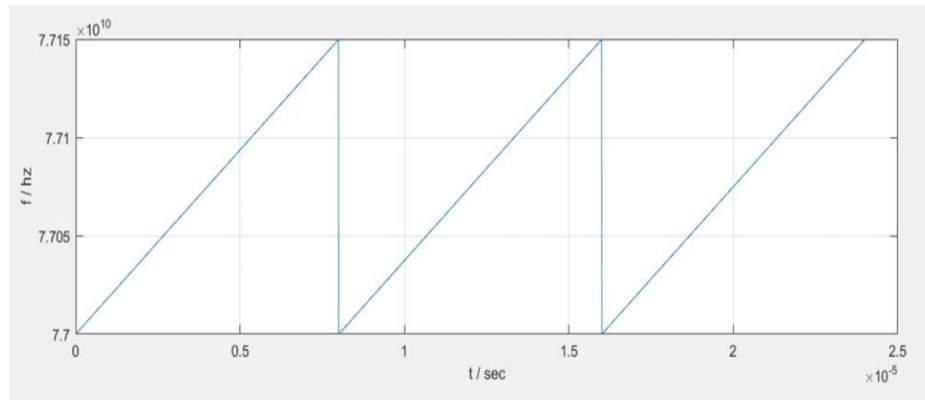
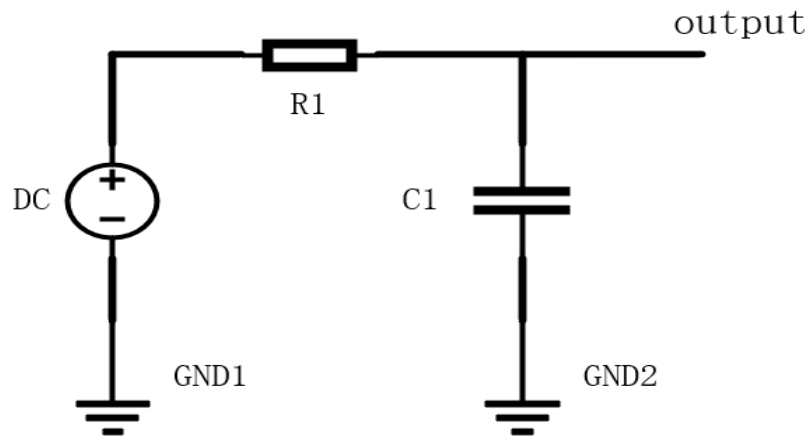
A chirp is characterized by a start frequency (f_c), Bandwidth(B) and duration(T_c). The Slope (K) of the chirp defines the rate at which the chirp ramps up. In this example the chirp is sweeping a bandwidth of 150MHz in 8us which corresponds to a Slope of 18.75MHz/us.

The number of the sampling times(N_r) is very important for the range limitation. And the length of the sample, the number of the chirp sampled(N_d), is also very important for velocity measurement and velocity resolution. The N_r of our radar is 1024. And the N_d of our radar is 128.

The frequency domain of the IF signal will tell us the distance of the object. And the phase domain will tell us the velocity of the object.

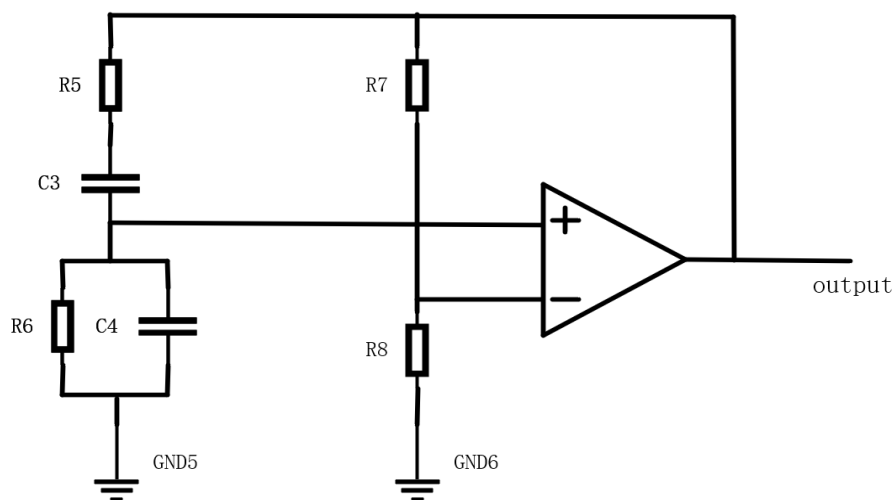
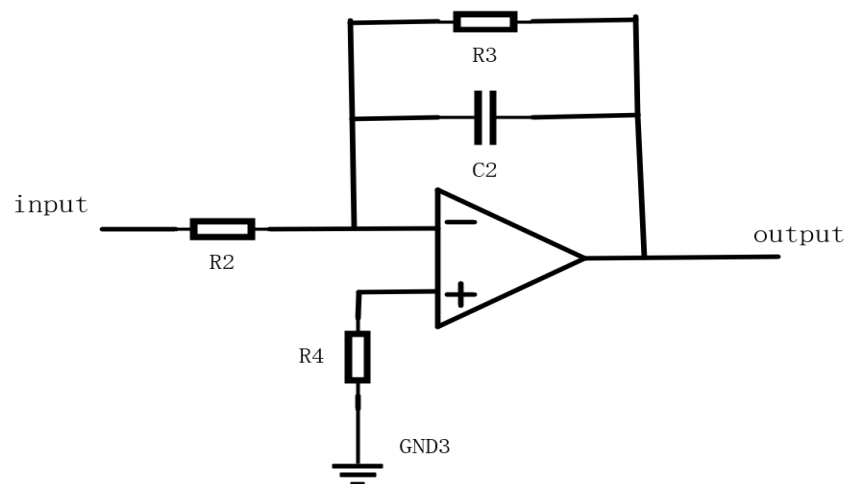
2 Analysis

2.1 Electric Circuit



From this circuit, we get the velocity of the shape of the function(1)

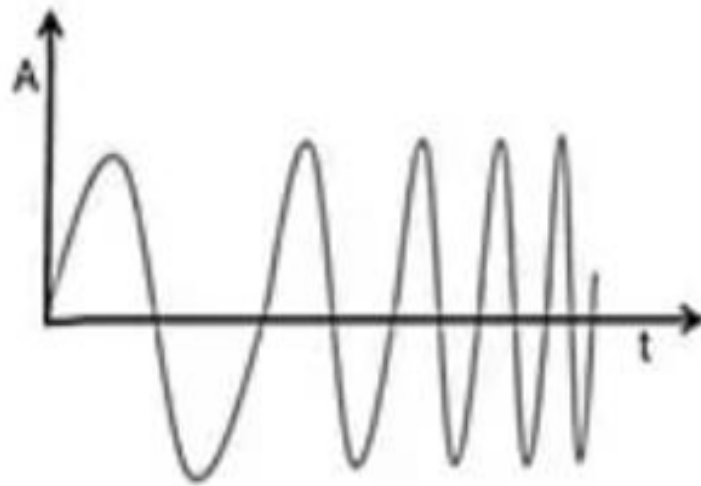
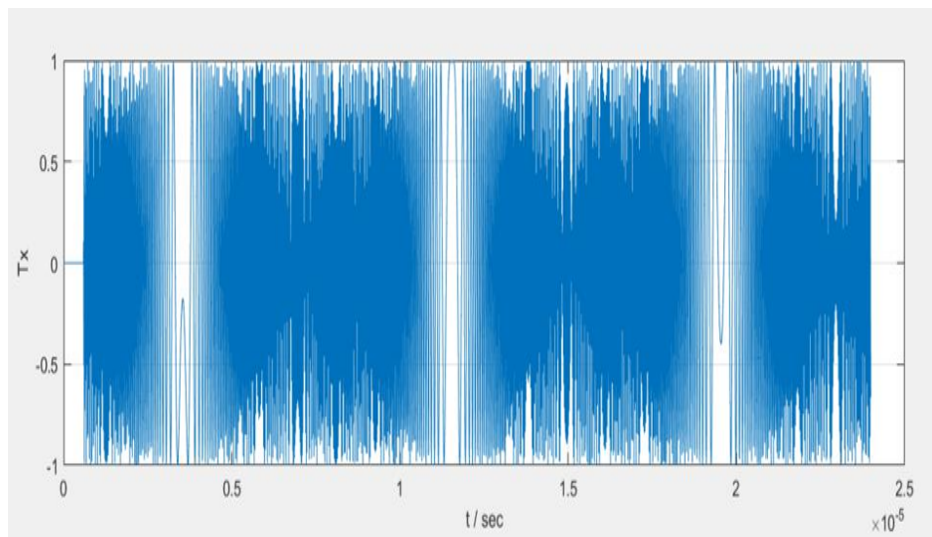
$$f(t) = f_c + k * t \quad (1)$$



And then use the integral circuit to get the phase of the equation(2). And then use the oscillator to get the cosine wave showed in equation(2)

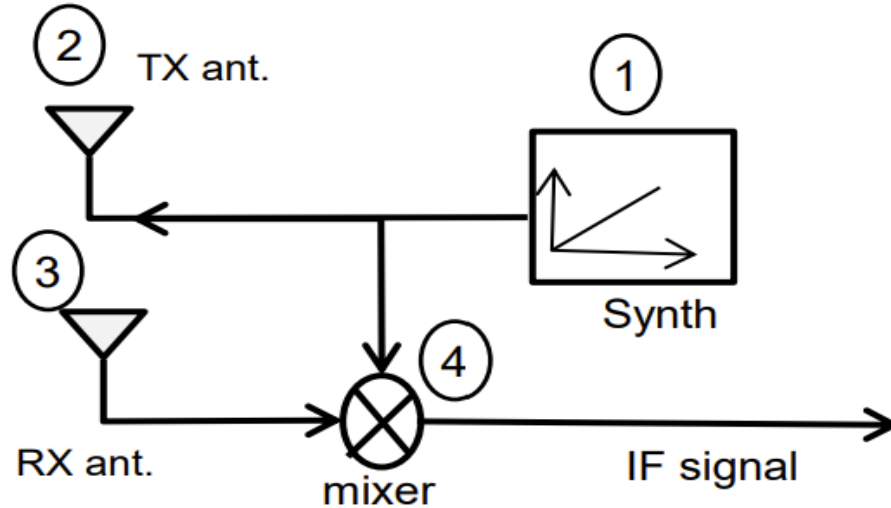
$$s(t) = \cos(2\pi(f_c t + Kt^2/2)) \quad (2)$$

The chirp signal is made. Its figures in time domain are showed in the follow plots.



2.2 Theory Analysis

The measurement of the speed and the distance needs the following system.



The distance measurement depends on the frequency of the IF signal.

$$\begin{aligned}
 x(t) &= \cos(\varphi_t(t)) \cos(\varphi_r(t)) \\
 &= \frac{1}{2} \cos(\varphi_t(t) + \varphi_r(t)) + \frac{1}{2} \cos(\varphi_t(t) - \varphi_r(t))
 \end{aligned}$$

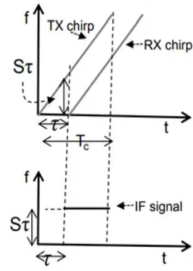
With low frequency sampling,
high frequency signal will be
ignored because of
under_sampling.

$$\begin{aligned}
 &\downarrow \\
 IF(t) &= A \cos(\varphi_t(t) - \varphi_r(t))
 \end{aligned}$$

$$IF(t) = A \cos(\varphi_t(t) - \varphi_r(t))$$

Calculation

$$\begin{aligned}\varphi_t(t) - \varphi_r(t) &= 2\pi \left(f_c t + \frac{K}{2} t^2 \right) - 2\pi \left(f_c (t - \tau) + \frac{K}{2} (t - \tau)^2 \right) \\ &= 2\pi f_c \tau + 2\pi K \tau t + \pi K \tau^2\end{aligned}$$



Reality

Fourier transform of cos

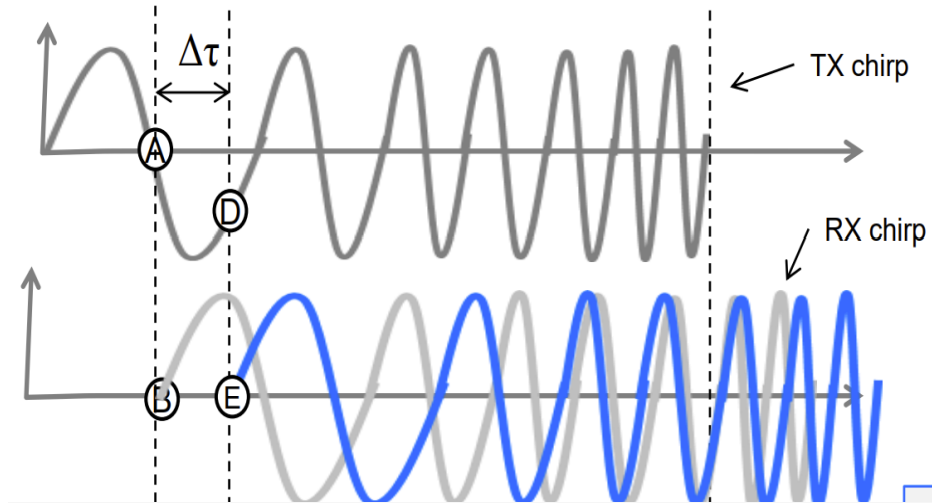
$$\cos \omega_0 t \quad \pi [\delta(\omega - \omega_0) + \delta(\omega + \omega_0)]$$

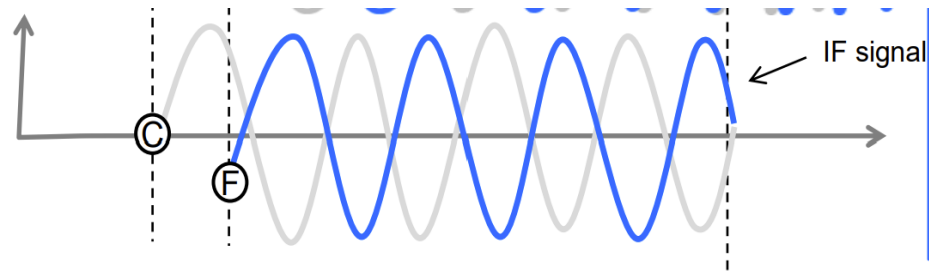
find ω_0

Get range resolution and limitation

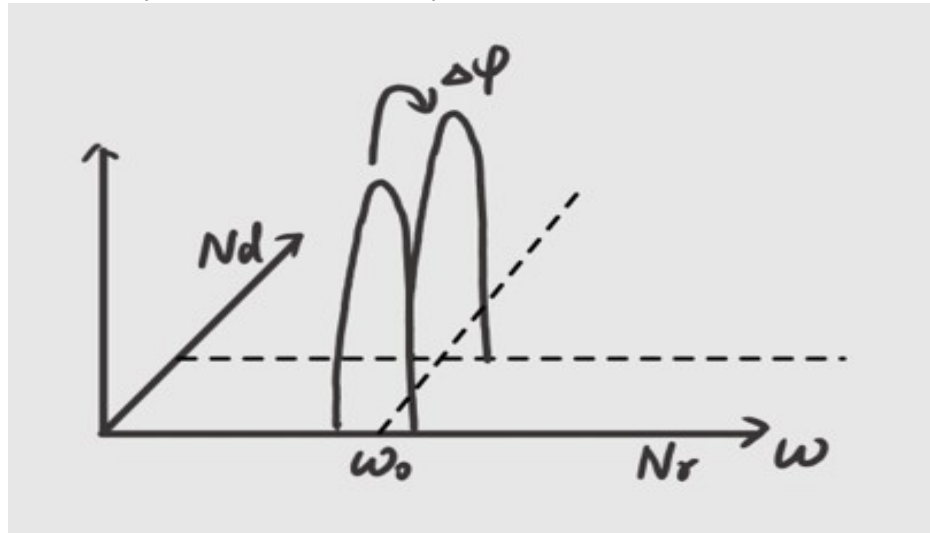
$$R = \frac{c}{2B} \times \frac{T f_s}{N_r} \times n$$

The measurement of the velocity depends on the phase. When the object moves a small distance between two continuous chirp periods, there will be a small change in frequency but a big change in phase. (If you use enough periods) Every two continuous chirps will have the same phase change, but their differences on amplitude are not the same. So we need to find the most obvious difference between two chirps in our sampling to calculate the velocity more clearly. The reason why there are phases changing showed in the follow plot.





And the way to calculate the velocity is the same as the distance.

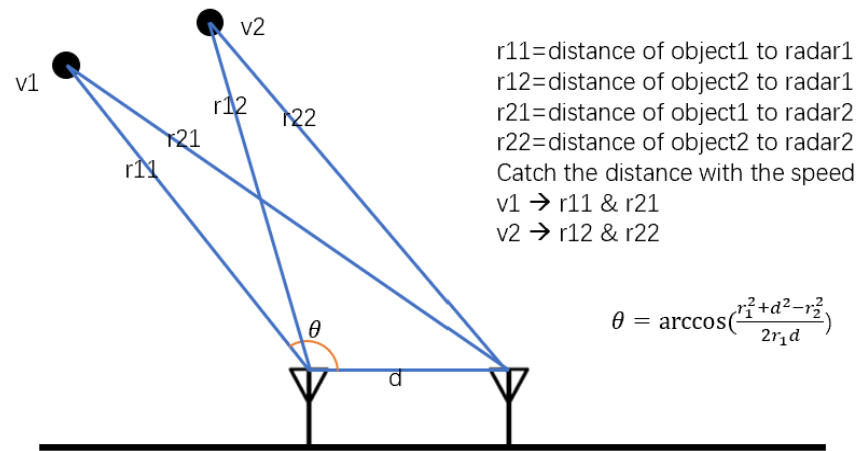


$$\omega = \frac{\partial \varphi}{\partial t} \quad \xrightarrow{\text{Velocity resolution}} \quad \text{Velocity of object}$$

$$\Delta v = \frac{c}{2f_c N_d T}$$

And the number of the objects does not influence the measurement of the distance and speed, unless the difference of both their speeds and distances are less than the resolution.

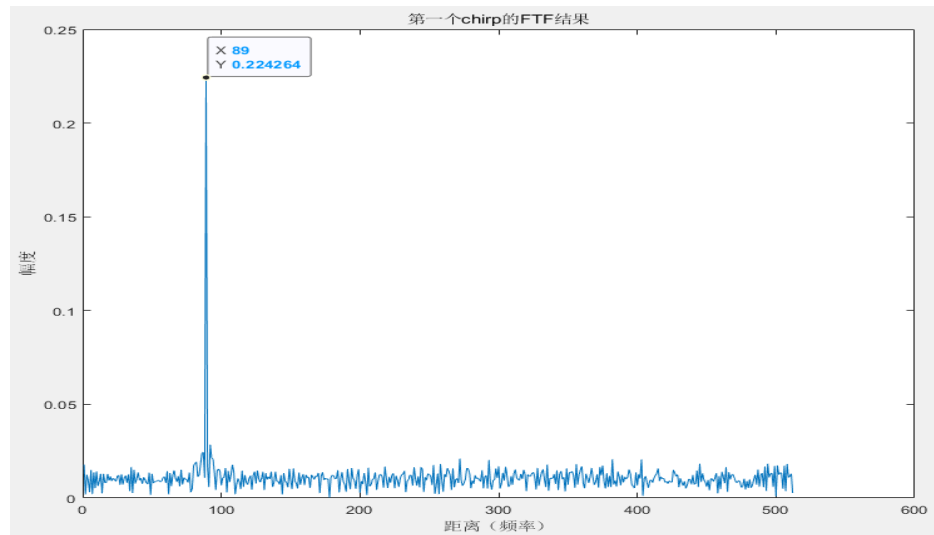
When it comes the positioning, the measurement of the angel is a big question for us. So we design the following system. We want to calculate the angel by using the cosine of the angel.



3 Plot And Result

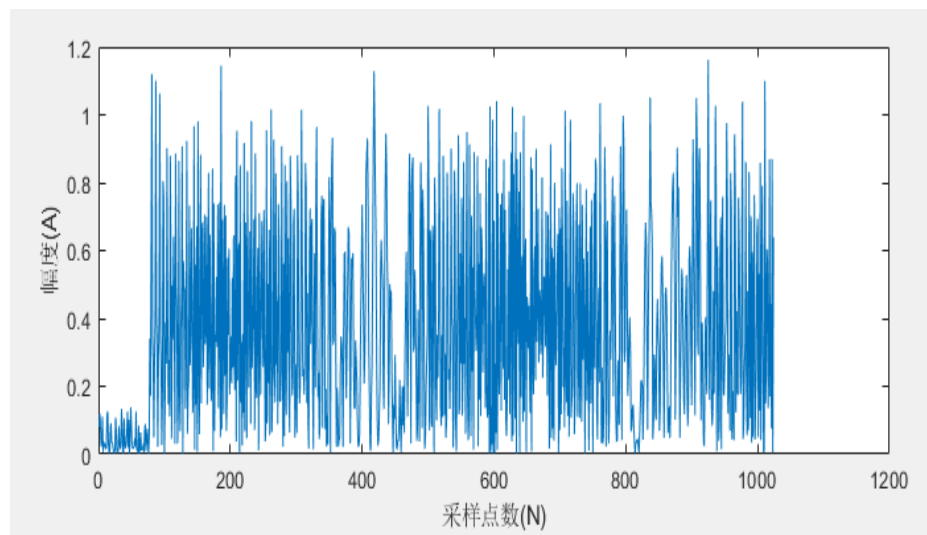
3.1 Question A

3.1.1 Range Result

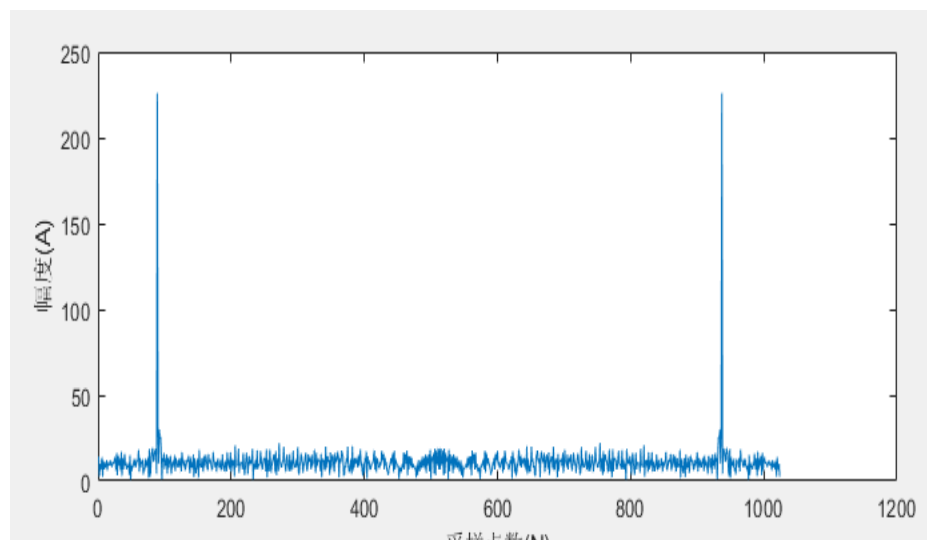


3.2 Question B

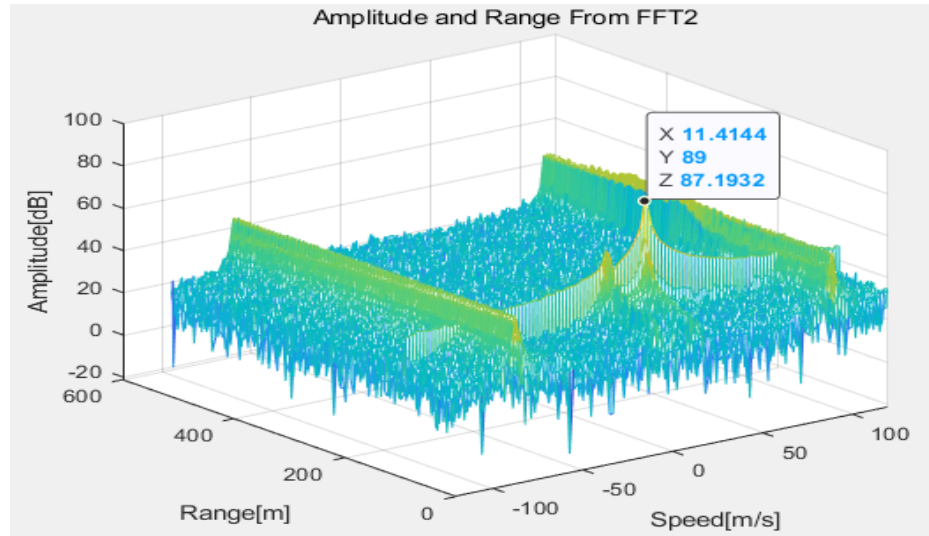
3.2.1 IF Signal



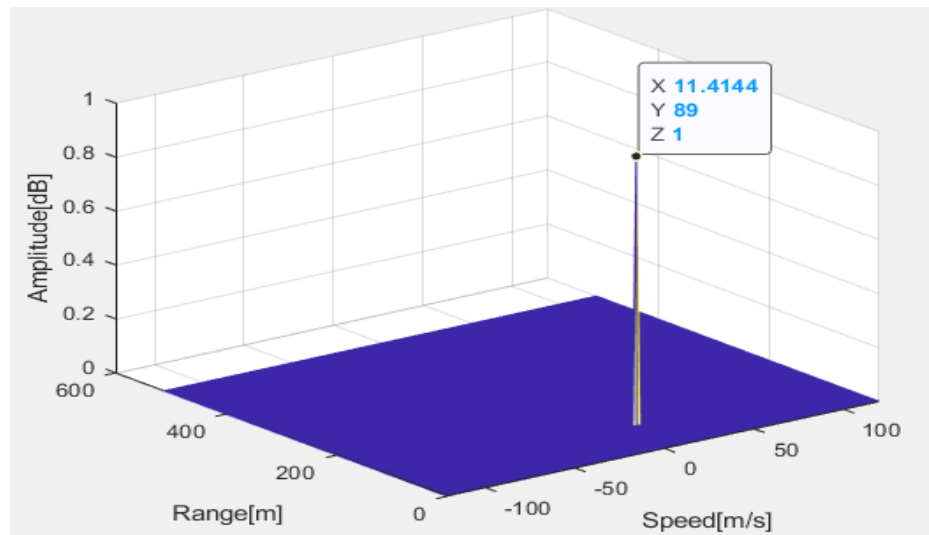
3.2.2 FFT Result



3.2.3 FFT2 Result



3.2.4 Velocity Result



3.3 Question C

3.3.1 Range Resolution

First Method:

$$\begin{aligned}
 R &= \frac{\tau c}{2} \\
 B &= K \times T \rightarrow f_m = K \times \tau \\
 R &= \frac{f_m c}{2K} \\
 \Delta R &= \frac{\Delta f_m c}{2S} \\
 \Delta f_m &= \frac{f_s}{N_r} \rightarrow f_s = \frac{N_r}{T} \rightarrow \Delta f_m = \frac{1}{T} \\
 \Delta R &= \frac{\Delta f_m c}{2K} = \frac{c}{2KT} = \frac{c}{2B} = \frac{3 \times 10^8}{2 \times 150 \times 10^6} m = 1m
 \end{aligned}$$

Second Method:

$$\begin{aligned}
 f_s &= \frac{N_r}{T} \\
 f_m &= \frac{n}{T} \\
 f_m &= \frac{f_s}{N} \times n \\
 R &= \frac{cTf_m}{2B} = \frac{cT}{2B} \times \frac{f_s}{N} \times n = \frac{c}{2B} \times \frac{Tf_s}{N_r} \times n = 1 \times 1 \times n \\
 &= 1 \times n \\
 \Delta R &= 1m
 \end{aligned}$$

3.3.2 Velocity Resolution

First Method:

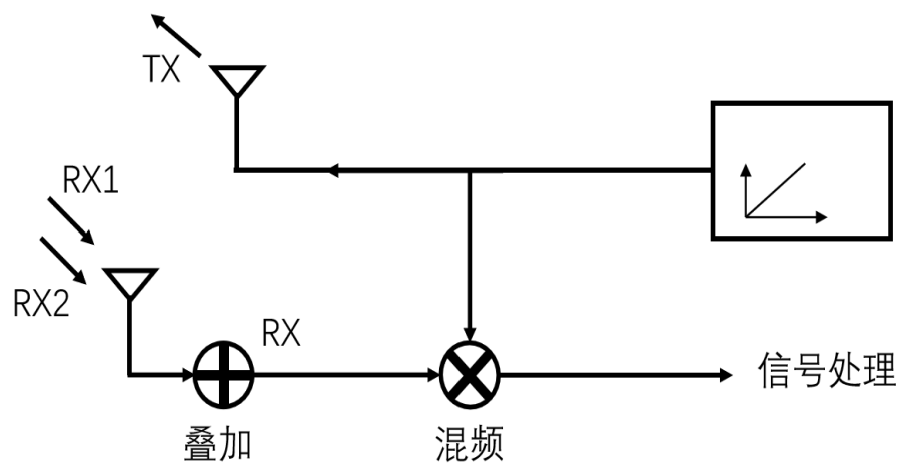
$$\begin{aligned}
 \phi_0 &= \frac{4\pi d}{\lambda} \\
 \text{Two Contiguous Periods } \Delta\phi &= \frac{4\pi vT}{\lambda} \\
 v &= \frac{\lambda \Delta\phi}{4\pi T} \\
 \Delta v &= \frac{\lambda}{2N_d T} = \frac{\frac{c}{f_c}}{2N_d T} = \frac{c}{2N_d T f_c} \\
 &= \frac{3 \times 10^8}{2 \times 128 \times 8 \times 10^{-6} \times 77 \times 10^9} = 1.9024m/s
 \end{aligned}$$

Second Method:

$$\begin{aligned}
 \omega &= \frac{4\pi f_c v T}{c} \\
 v &= \frac{\omega c}{4\pi f_c T} \\
 F_s &= \frac{N_d}{N_d \times T} = \frac{1}{T} \rightarrow \Delta f = \frac{F_s}{N_d} \\
 \Delta\omega &= 2\pi \frac{\Delta f}{F_s} = \frac{2\pi}{N_d} \\
 \Delta v &= \frac{\Delta\omega \times c}{4\pi f_c T} = \frac{c}{2f_c N_d T} = 1.9024m/s
 \end{aligned}$$

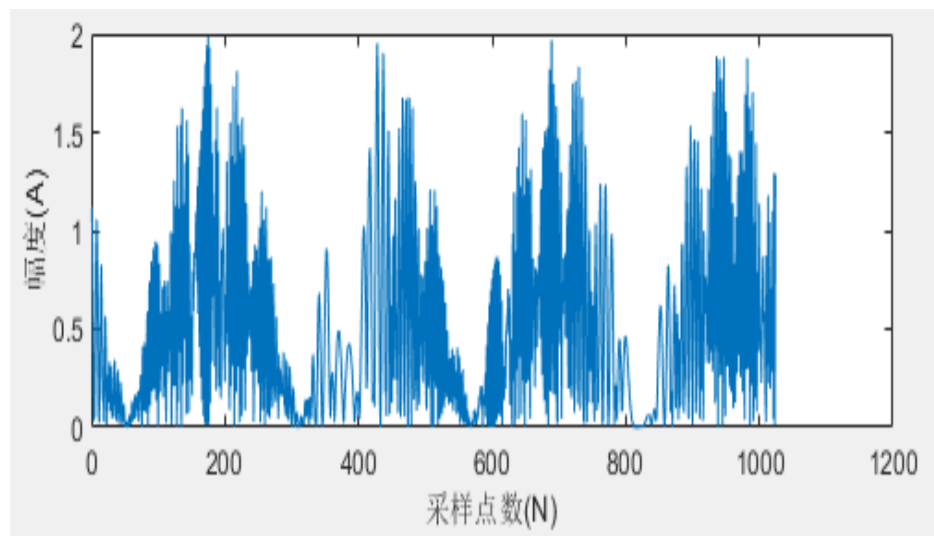
3.4 Question D

3.4.1 System

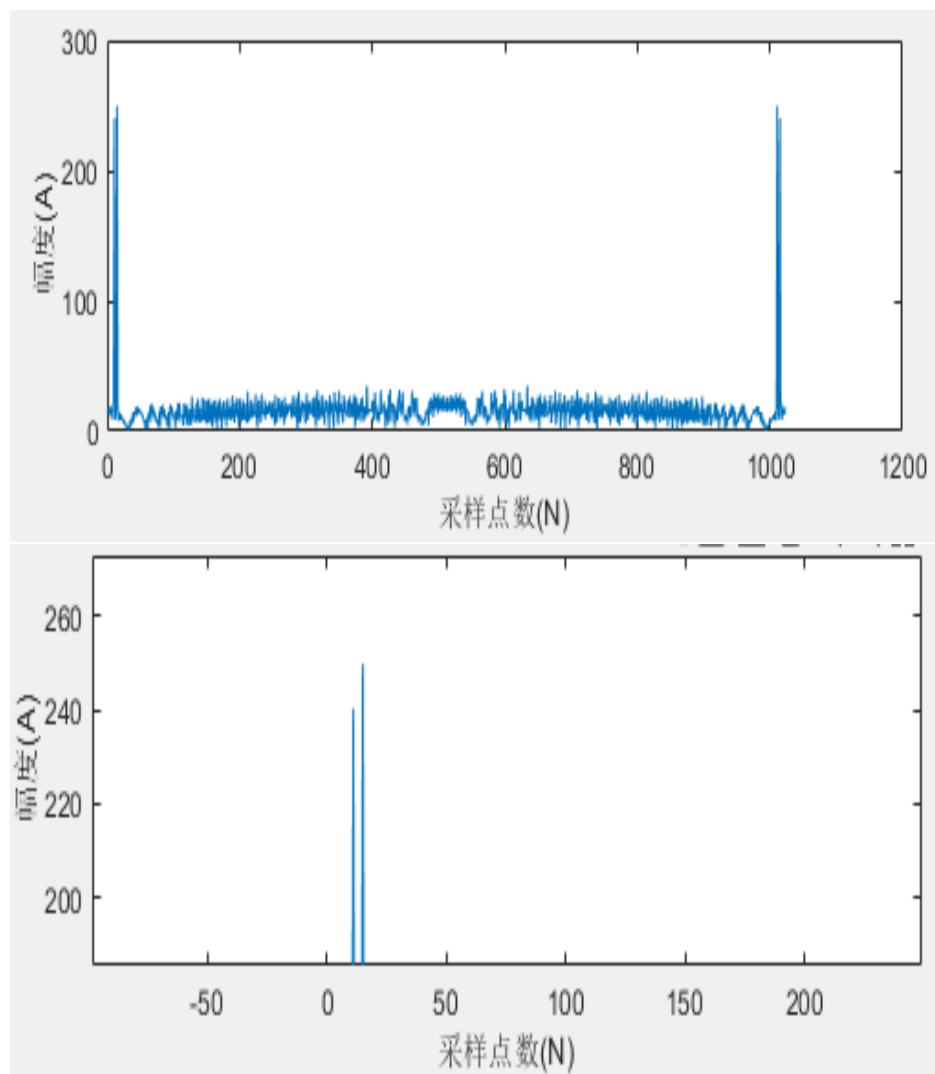


物体1			物体2		
	实际	计算		实际	计算
距离	14	14	距离	10	10
速度	10	11.4144	速度	14	15.2192
		朝上取精度			朝上取精度

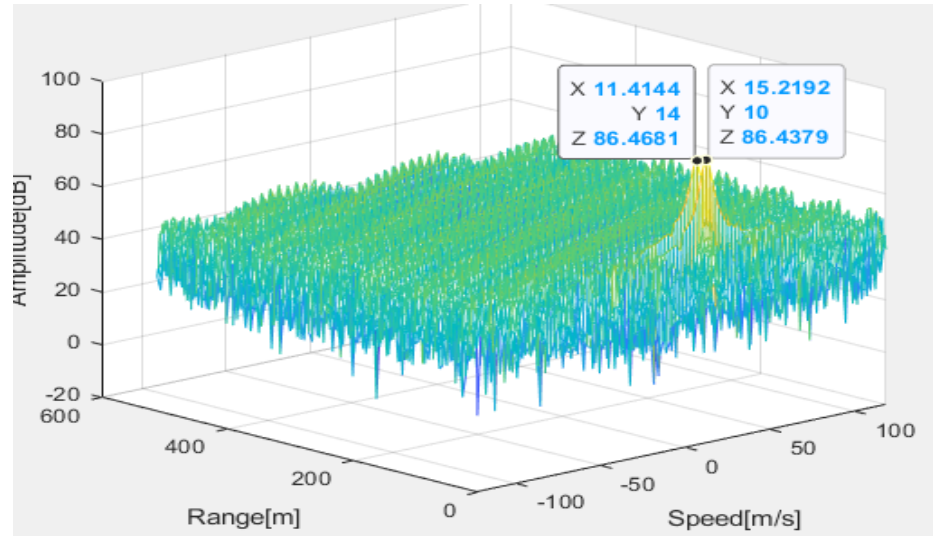
3.4.2 IF Signal



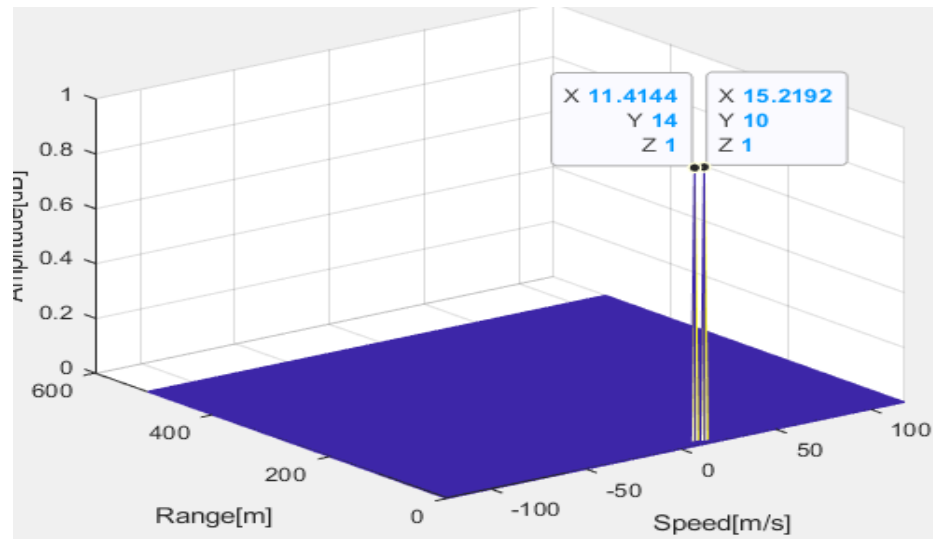
3.4.3 FFT Result



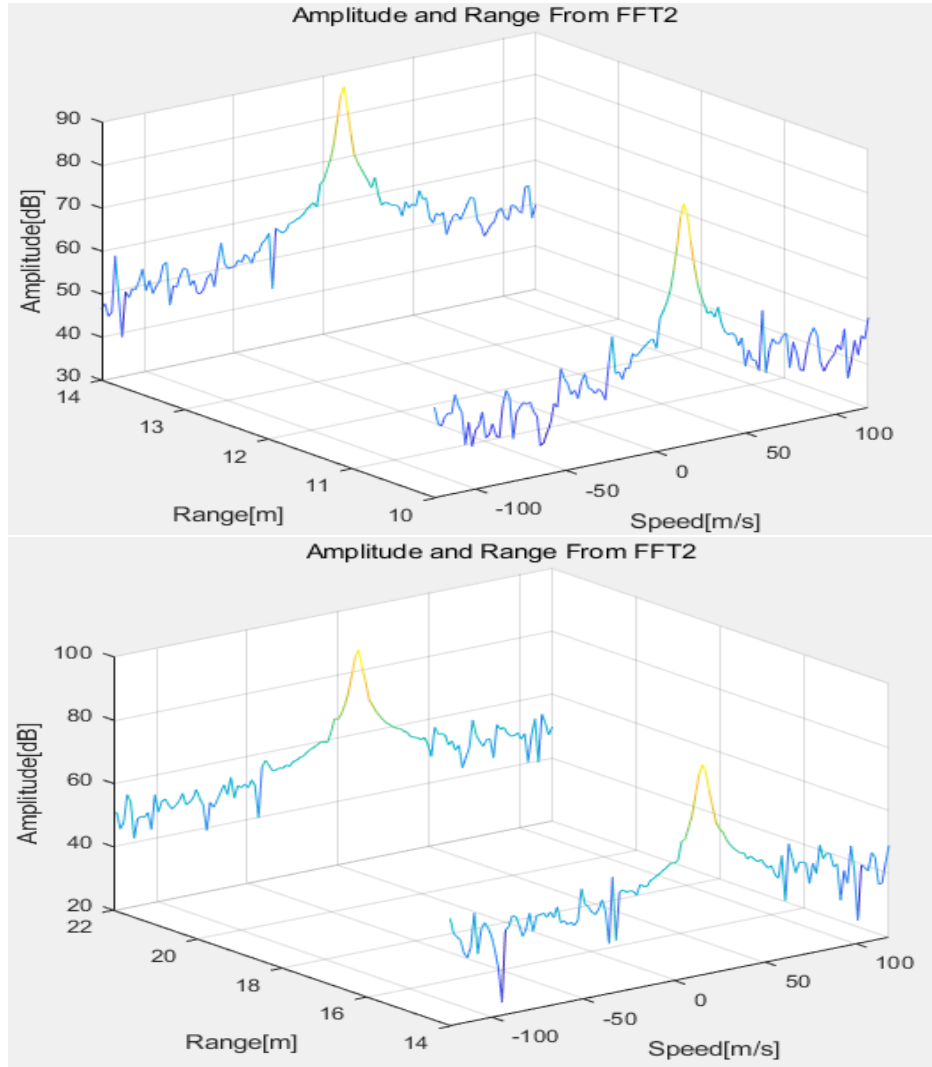
3.4.4 FFT2 Result



3.4.5 Velocity Result



3.5 Question E



4 Reflection

There are two main lessons we learned from this project.

The first one is the theory analysis and the reality situation are not enough for our analysis. The bridge between the sampling of the reality and the calculation is also an important factor for our analysis. Just like the distance resolution of our radar. It is influenced by the F_s , the frequency of the sampling. In other word, we can not rebuild the signal in computer. We can only approximate the real signal.

The second one is the use of the phase. When we have an idea about a question but the calculation can not make our thoughts come true. We should consider a new way to deal with it. The frequency differences between different samples can not be showed by the computer, because it is too small to resolute. But the differences of phase is obvious enough for the computer to show it for us. That is a new and suitable way to solve this problem.

5 Extension

The equation of the range limitation in real world.

$$\text{SNR} = \frac{\overbrace{P_t G_t (\text{RCS}) G_r}^{\text{Rx signal level}} \left(\frac{\lambda^2}{4\pi} \right) T_f}{(4\pi R^2) (4\pi R^2) \underbrace{(kT)(NF)}_{\text{Noise level}}}$$

(linear scale)

The equation of the distance resolution in real world.

$$\sigma_R = \frac{\Delta R}{1.8 \sqrt{2 \text{SNR}}}$$

The radar in the real world need to rotate. In this way, we can get both range limitation and the distance resolution. The efficiency is high and its directivity is very good.

The high frequency ADC core is very expensive. The resampling can made more sampling points, canwe use it to replace the high frequency ADC core?

6 Code

6.1 Question A

```
code:
fc = 77e9;
B = 150*106;
Tchirp = 8 * 10(-6);
K = B/Tchirp;
c = 3e8;
Nr = 1024;
Nd = 128;
range_res = 1;
signal = reshape(x1, Nr, Nd);
sig_fft = fft(x1(1 : 1024))./Nr;
sig_fft = abs(sig_fft);
sig_fft = sig_fft(1 : (Nr/2));
figure;
plot(sig_fft);
xlabel('Range frequency');
ylabel('Amplitude')
title('The FFT of the first chirp')
```

6.2 Question B

code:

```
fc = 77*109;
B = 150 * 106;
Tchirp = 8 * 10(-6);
K = B/Tchirp;
c = 3e8;
lamuda = c/fc;
range_res = 1;
Nr = 1024;
Nd = 128;
vel_res = lamuda/(2 * Tchirp * Nd);
Mix_fft1d = reshape(x2, [Nr, Nd]);
(InitialSignal)
figure(1);
subplot(2, 1, 1);
plot(abs((Mix_fft1d(:, 1))));
xlabel('Samplingpoint(N)')
ylabel('Amplitude(A)')
subplot(2, 1, 2);
plot(abs(fft(Mix_fft1d(:, 1))));
xlabel('Samplingpoint(N)')
ylabel('Amplitude(A)')
```

```
(Twoffftsetuponefft2
signal_fft = fft(Mix_fft1d, Nr);
figure(2)
mesh(db(abs(signal_fft./max(signal_fft))));
title('RangefromFirstFFT');
ylabel('Range[m]');
xlabel('chirps[N]')
zlabel('Amplitude(dB)');
title('FrequencyFFT')
for i = 1 : Nr
    signal_fft2(i, :) = fftshift(fft(signal_fft(i, :)));
end
signal_fft2 = fft2(Mix_fft1d, 1024, 128);)
```

```
for i = 1 : Nr/2
    (Velocityexistspositiveandnegativepartsssoapplyfftshifttovelocity; Rangeiscutintohalfpert)
    signal_fft3(i, :) = fftshift(signal_fft2(i, :));
end
doppler_axis = vel_res * ((-Nd/2 : Nd/2 - 1));
```

```

range_axis = range_res * ((1 : Nr/2));
figure(3);
mesh(doppler_axis, range_axis, db(abs(signal_fft3)));
xlabel('Speed[m/s]');
ylabel('Range[m]');
zlabel('Amplitude[dB]');
title('AmplitudeandRangeFromFFT2');
(CFARimplementation)RDM = (abs(signal_fft3));
n_train_cells = 8;
n_train_bands = 8;
(test(CUT)foraccurateestimation)n_guard_cells = 4;
n_guard_bands = 4;
offset = 1.4;
noise_level = zeros(1,1);
RDM = RDM/max(RDM(:));
RDM(RDM == 0 | RDM == 1) = 0;
figure('Name','CA - CFARFilteredRDM')
mesh(doppler_axis, range_axis, RDM);
title('CA - CFARFilteredRDMsurfaceplot');
xlabel('Speed[m/s]');
ylabel('Range[m]');
zlabel('Amplitude[dB]');
(11.4144m/s; 89m)
(1 - 512m)
(-1.9024 * 64 1.9024 * 63 = -121.7532 119.8509)

```


6.3 Question D

```
code:
(Radar Specifications)
fc = 77*109;
B = 150 * 106;
Tchirp = 8 * 10(-6);
K = B/Tchirp;
c = 3e8;
lamuda = c/fc;
rangeres = 1;
Nr = 1024;
Nd = 128;
velres = lamuda/(2 * Tchirp * Nd);
(User Defined Range and Velocity of target)
range = 14;
vel = 10;
range1 = 10;
vel1 = 14;
maxvel = lamuda/(4 * Tchirp); (m/s)
(FMCW Waveform Generation)
B = c/(2 * rangeres);
slope = B/Tchirp;
t = linspace(0, Nd * Tchirp, Nr * Nd);
(total time for samples)
rt = zeros(1, length(t));
td = 2 * (range + vel * t)/c;
td1 = 2 * (range1 + vel1 * t)/c;
Tx = cos(2 * pi * (fc * t + (slope * t.*t)/2));
Rx = cos(2 * pi * (fc * (t - td) + (slope * (t - td). * (t - td))/2));
Rx1 = cos(2 * pi * (fc * (t - td1) + (slope * (t - td1). * (t - td1))/2));

(Mix = Tx. * Rx;
Mix frequency
Mix1 = Tx. * Rx1;
Mix = Mix + Mix1;)
Mix = Tx. * (Rx + Rx1);
Mixfft1d = Mix + awgn(Mix, 20); (Added Gaussian white noise)

Mixfft1d = reshape(Mix, [Nr, Nd]);
(Initial signal)
figure(1);
subplot(2, 1, 1);
plot(abs((Mixfft1d(:, 1))));
```

```

xlabel('Samplingpoint(N)')
ylabel('Amplitude(A)')
subplot(2,1,2);
plot(abs(fft(Mix_fft1d(:,1))));
xlabel('Samplingpoint(N)')
ylabel('Amplitude(A)')
    pks1,locs1
= findpeaks(abs(fft(Mix_fft1d(:,1))));
cnt = 1;
fori = 1 : length(locs1)
if(pks1(i) > max(pks1) * 0.9locs1(i) <= 512)
locs2(cnt) = locs1(i);
cnt = cnt + 1;
end
end
signal_fft = fft(Mix_fft1d, Nr);
figure(2)
mesh(db(abs(signal_fft./max(signal_fft))));
title('RangefromFirstFFT');
ylabel('Range[m]');
xlabel('chirps[N]')
zlabel('Amplitude(dB)');
title('FrequencyFFT')
signal_fft2 = zeros(512,128);
fori = 1 : Nr/2
signal_fft2(i,:) = fftshift(fft(signal_fft(i,:)));
end

(for i = 1 : length(locs2)
signal_fft2(locs2(i),:) = fftshift(fft(signal_fft(locs2(i),:)));
end
signal_fft2 = fft2(Mix_fft1d,1024,128);
fori = 1 : Nr/2
(Velocityexistspositiveandnegativepartsssoapplyfftshifttovelocity;Rangeiscutintohalfpert)
signal_fft3(i,:) = fftshift(signal_fft2(i,:));
end)

doppler_axis = vel_res * ((-Nd/2 : (Nd - 1)/2) - 1);
range_axis = range_res * ((1 : Nr/2) - 1);
figure(3);
mesh(doppler_axis, range_axis, db(abs(signal_fft2)));
xlabel('Speed[m/s]');
ylabel('Range[m]');
zlabel('Amplitude[dB]');

```

```

title('AmplitudeandRangeFromFFT2');
(CFARimplementation)
RDM = db(abs(signal_fft2));
n_train_cells = 8;
n_train_bands = 8;
(test(CUT)foraccurateestimation)
n_guard_cells = 4;
n_guard_bands = 4;
offset = 1.4;
noiselevel = zeros(1,1);
for i = 1 : length(locs2)
    ,v_r(i)
    = max(RDM(locs2(i),:));
    RDM(locs2(i),:) = 0;
    RDM(locs2(i),v_r(i)) = 1;
end
for i = 1 : Nr/2
    if(i == locs2(1) || i == locs2(2))
        RDM(i,:) = 0;
    end
end
v_r = (v_r - 65 - 1) * vel_res;
r_r = locs2 - 1;

(RDM(RDM == 0 || RDM == 1) = 0;
RDM(RDM < 0.98) = 0;
count = 0;
for i = 1 : 512
    for j = 1 : 128
        if(RDM(i,j) == 0)
            count = count + 1;
        end
    end
end)

figure('Name','CA - CFARFilteredRDM')
mesh(doppler_axis, range_axis, RDM);
title('CA - CFARFilteredRDMsurfaceplot');
xlabel('Speed[m/s]');
ylabel('Range[m]');
zlabel('Amplitude[dB]');
(ending)

```

6.4 Question E

```
code:
(Radar Specifications)
fc = 77*109;
B = 150 * 106;
Tchirp = 8 * 10(-6);
K = B/Tchirp;
c = 3e8;
lamuda = c/fc;
rangeres = 1;
Nr = 1024;
Nd = 128;
velres = lamuda/(2 * Tchirp * Nd);
leidad = 10;
(UserDefinedRangeandVelocityoftargetRadar1)
range11 = 14;
vel11 = 10;
range21 = 10;
vel21 = 14;
maxvel = lamuda/(4 * Tchirp); (m/s)
(FMCWWaveformGeneration)
B = c/(2 * rangeres);
slope = B/Tchirp;
t = linspace(0, Nd * Tchirp, Nr * Nd); (totaltimeforsamples)
rt = zeros(1, length(t));
td = 2 * (range11 + vel11 * t)/c;
td1 = 2 * (range21 + vel21 * t)/c;
Tx = cos(2 * pi * (fc * t + (slope * t. * t)/2));
Rx = cos(2 * pi * (fc * (t - td) + (slope * (t - td). * (t - td))/2));
Rx1 = cos(2 * pi * (fc * (t - td1) + (slope * (t - td1). * (t - td1))/2));
Mix = Tx. * Rx; (Mixfrequency)
Mix1 = Tx. * Rx1;
Mix = Mix + Mix1;
(Mixfft1d = Mix + awgn(Mix, 20); AddedGaussianwhitenoise)
Mixfft1d = reshape(Mix, [Nr, Nd]);
(Initialsignal)
figure(1);
subplot(2, 1, 1);
plot(abs((Mixfft1d(:, 1))));
xlabel('Samplingpoint(N)')
ylabel('Amplitude(A)')
subplot(2, 1, 2);
plot(abs(fft(Mixfft1d(:, 1))));
xlabel('Samplingpoint(N)')
```

```

ylabel('Amplitude(A)')
    pks1,locs1
= findpeaks(abs(fft(Mix_fft1d(:,1))));
cnt = 1;
fori = 1 : length(locs1)
if(pks1(i) > max(pks1) * 0.9locs1(i) <= 512)
locs2(cnt) = locs1(i);
cnt = cnt + 1;
end
end
signal_fft = fft(Mix_fft1d, Nr);
figure(2)
mesh(db(abs(signal_fft./max(signal_fft))));
title('RangefromFirstFFT');
ylabel('Range[m]');
xlabel('chirps[N]')
zlabel('Amplitude(dB)');
title('FrequencyFFT')
signal_fft2 = zeros(512, 128);
fori = 1 : length(locs2)
signal_fft2(locs2(i), :) = fftshift(fft(signal_fft(locs2(i), :)));
end

```

```

(signal_fft2 = fft2(Mix_fft1d, 1024, 128);
fori = 1 : Nr/2
(Velocityexistspositiveandnegativepartsoapplyfftshifttovelocity; Rangeiscutintohalffpart)
signal_fft3(i, :) = fftshift(signal_fft2(i, :));
end)

```

```

doppler_axis = vel_res * ((-Nd/2 : (Nd - 1)/2) - 1);
range_axis = range_res * ((1 : Nr/2) - 1);
figure(3);
mesh(doppler_axis, range_axis, db(abs(signal_fft2)));
xlabel('Speed[m/s]');
ylabel('Range[m]');
zlabel('Amplitude[dB]');
title('AmplitudeandRangeFromFFT2');
(CFARimplementation)
RDM = db(abs(signal_fft2));
n_train_cells = 8;
n_train_bands = 8;
(test(CUT)foraccurateestimation)
n_guard_cells = 4;
n_guard_bands = 4;

```

```

offset = 1.4;
noiselevel = zeros(1,1);
for i = 1 : length(locs2)
    ,vr1(i)
    = max(RDM(locs2(i),:));
    RDM(locs2(i),:) = 0;
    RDM(locs2(i),vr1(i)) = 1;
end
vr1 = (vr1 - 65 - 1) * velres;
rr1 = locs2 - 1;
(UserDefinedRangeandVelocityoftargetradar2)
range12 = 22;
vel12 = 10;
range22 = 14;
vel22 = 14;
maxvel = lamuda/(4 * Tchirp); (m/s)
(FMCWWaveformGeneration)
B = c/(2 * rangeres);
slope = B/Tchirp;
t = linspace(0, Nd * Tchirp, Nr * Nd);
(totaltimesamples)
rt = zeros(1, length(t));
td = 2 * (range12 + vel12 * t)/c;
td1 = 2 * (range22 + vel22 * t)/c;
Tx = cos(2 * pi * (fc * t + (slope * t. * t)/2));
Rx = cos(2 * pi * (fc * (t - td) + (slope * (t - td). * (t - td))/2));
Rx1 = cos(2 * pi * (fc * (t - td1) + (slope * (t - td1). * (t - td1))/2));
Mix = Tx. * Rx;
(Mixfrequency)
Mix1 = Tx. * Rx1;
Mix = Mix + Mix1;
(Mixfft1d = Mix + awgn(Mix, 20); AddedGaussianwhitenoise)
Mixfft1d = reshape(Mix, [Nr, Nd]);
(Initialsignal)
figure(1);
subplot(2, 1, 1);
plot(abs((Mixfft1d(:, 1))));
xlabel('Samplingpoint(N)')
ylabel('Amplitude(A)')
subplot(2, 1, 2);
plot(abs(fft(Mixfft1d(:, 1))));
xlabel('Samplingpoint(N)')
ylabel('Amplitude(A)')
pks1, locs1
= findpeaks(abs(fft(Mixfft1d(:, 1))));
cnt = 1;

```

```

fori = 1 : length(locs1)
if(pks1(i) > max(pks1) * 0.9locs1(i) <= 512)
locs2(cnt) = locs1(i);
cnt = cnt + 1;
end
end
signal_fft = fft(Mix_fft1d, Nr);
figure(2)
mesh(db(abs(signal_fft./max(signal_fft))));
title('RangeFromFirstFFT');
ylabel('Range[m]');
xlabel('chirps[N]')
zlabel('Amplitude[dB]');
title('FrequencyFFT')
signal_ft2 = zeros(512, 128);
fori = 1 : length(locs2)
signal_ft2(locs2(i), :) = fftshift(fft(signal_fft(locs2(i), :)));
end

```

```

(signal_ft2 = fft2(Mix_fft1d, 1024, 128);
fori = 1 : Nr/2
(fftshift)
signal_ft3(i, :) = fftshift(signal_ft2(i, :));
end)

```

```

doppler_axis = vel_res * ((-Nd/2 : (Nd - 1)/2) - 1);
range_axis = range_res * ((1 : Nr/2) - 1);
figure(3);
mesh(doppler_axis, range_axis, db(abs(signal_ft2)));
xlabel('Speed[m/s]');
ylabel('Range[m]');
zlabel('Amplitude[dB]');
title('AmplitudeandRangeFromFFT2');
(CFARimplementation)
RDM = db(abs(signal_ft2));
n_train_cells = 8;
n_train_bands = 8;
(test(CUT)foraccurateestimation)
n_guard_cells = 4;
n_guard_bands = 4;
offset = 1.4;
noise_level = zeros(1, 1);
fori = 1 : length(locs2)
    ,vr2(i)

```

```

= max(RDM(locs2(i),:));
RDM(locs2(i),:) = 0;
RDM(locs2(i),v_r2(i)) = 1;
end
v_r2 = (v_r2 - 65 - 1) * vel_r es;
r_r2 = locs2 - 1;
num = length(v_r1);
leida1 = sortrows([v_r1;r_r1]');
leida2 = sortrows([v_r2;r_r2]');
leida11 = leida1(:,2)';
leida22 = leida2(:,2)';
for i = 1 : num
    (Basedontheradar1)
    angle(i) = acos((leida11(i)^2 + leidad^2 - leida22(i)^2)/(2 * leida11(i) * leidad));
end
leida = leida11;
disp(leida);
disp(angle);

```